In [1]:
```
import pandas as pd
```

In [2]:
```
dataset=pd.read_csv("Placement.csv")
```

In [3]:
```
dataset
```

Out[3]:

	sl_no	gender	ssc_p	ssc_b	hsc_p	hsc_b	hsc_s	degree_p	degree_t	workex	etest_p	specialisation	mba_p	status	salary
0	1	M	67.00	Others	91.00	Others	Commerce	58.00	Sci&Tech	No	55.0	Mkt&HR	58.80	Placed	270000.0
1	2	M	79.33	Central	78.33	Others	Science	77.48	Sci&Tech	Yes	86.5	Mkt&Fin	66.28	Placed	200000.0
2	3	M	65.00	Central	68.00	Central	Arts	64.00	Comm&Mgmt	No	75.0	Mkt&Fin	57.80	Placed	250000.0
3	4	M	56.00	Central	52.00	Central	Science	52.00	Sci&Tech	No	66.0	Mkt&HR	59.43	Not Placed	NaN
4	5	M	85.80	Central	73.60	Central	Commerce	73.30	Comm&Mgmt	No	96.8	Mkt&Fin	55.50	Placed	425000.0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
210	211	M	80.60	Others	82.00	Others	Commerce	77.60	Comm&Mgmt	No	91.0	Mkt&Fin	74.49	Placed	400000.0
211	212	M	58.00	Others	60.00	Others	Science	72.00	Sci&Tech	No	74.0	Mkt&Fin	53.62	Placed	275000.0
212	213	M	67.00	Others	67.00	Others	Commerce	73.00	Comm&Mgmt	Yes	59.0	Mkt&Fin	69.72	Placed	295000.0
213	214	F	74.00	Others	66.00	Others	Commerce	58.00	Comm&Mgmt	No	70.0	Mkt&HR	60.23	Placed	204000.0
214	215	M	62.00	Central	58.00	Others	Science	53.00	Comm&Mgmt	No	89.0	Mkt&HR	60.22	Not Placed	NaN

215 rows × 15 columns

In [4]:
```
dataset.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 215 entries, 0 to 214
Data columns (total 15 columns):
sl_no             215 non-null int64
gender            215 non-null object
ssc_p             215 non-null float64
ssc_b             215 non-null object
hsc_p             215 non-null float64
hsc_b             215 non-null object
hsc_s             215 non-null object
degree_p          215 non-null float64
degree_t          215 non-null object
workex            215 non-null object
etest_p           215 non-null float64
specialisation    215 non-null object
mba_p             215 non-null float64
status            215 non-null object
salary            148 non-null float64
dtypes: float64(6), int64(1), object(8)
memory usage: 25.3+ KB
```

In [5]:
```
#Quan=["ssc_p","hsc_p","etest_p","degree_p","mba_p","salary"]
#Qual=["gender","ssc_b","hsc_b","degree_t"]
```

In [6]:
```
dataset.columns
```

Out[6]:
```
Index(['sl_no', 'gender', 'ssc_p', 'ssc_b', 'hsc_p', 'hsc_b', 'hsc_s',
       'degree_p', 'degree_t', 'workex', 'etest_p', 'specialisation', 'mba_p',
       'status', 'salary'],
      dtype='object')
```

In [7]:
```
dir(dataset)
```

Out[7]:
```
['T',
 '_AXIS_ALIASES',
 '_AXIS_IALIASES',
 '_AXIS_LEN',
 '_AXIS_NAMES',
 '_AXIS_NUMBERS',
 '_AXIS_ORDERS',
 '_AXIS_REVERSED',
 '__abs__',
 '__add__',
 '__and__',
 '__array__',
 '__array_priority__',
 '__array_wrap__',
 '__bool__',
 '__class__',
 '__contains__',
 '__copy__',
 '__deepcopy__',
 '__delattr__',
 '__delitem__',
 '__dict__',
 '__dir__',
 '__div__',
 '__doc__',
 '__eq__',
 '__finalize__',
 '__floordiv__',
 '__format__',
 '__ge__',
 '__getattr__',
 '__getattribute__',
 '__getitem__',
 '__getstate__',
 '__gt__',
 '__hash__',
 '__iadd__',
 '__iand__',
 '__ifloordiv__',
 '__imod__',
 '__imul__',
 '__init__',
 '__init_subclass__',
 '__invert__',
 '__ior__',
 '__ipow__',
 '__isub__',
 '__iter__',
 '__itruediv__',
 '__ixor__',
 '__le__',
 '__len__',
 '__lt__',
 '__matmul__',
 '__mod__',
 '__module__',
 '__mul__',
 '__ne__',
 '__neg__',
 '__new__',
 '__nonzero__',
 '__or__',
 '__pos__',
 '__pow__',
 '__radd__',
 '__rand__',
 '__rdiv__',
 '__reduce__',
 '__reduce_ex__',
 '__repr__',
 '__rfloordiv__',
 '__rmatmul__',
 '__rmod__',
 '__rmul__',
 '__ror__',
 '__round__',
 '__rpow__',
 '__rsub__',
 '__rtruediv__',
 '__rxor__',
 '__setattr__',
 '__setitem__',
 '__setstate__',
 '__sizeof__',
 '__str__',
 '__sub__',
 '__subclasshook__',
 '__truediv__',
 '__weakref__',
 '__xor__',
 '_accessors',
 '_add_numeric_operations',
 '_add_series_only_operations',
 '_add_series_or_dataframe_operations',
 '_agg_by_level',
 '_agg_examples_doc',
 '_agg_summary_and_see_also_doc',
 '_aggregate',
 '_aggregate_multiple_funcs',
 '_align_frame',
 '_align_series',
 '_box_col_values',
 '_box_item_values',
 '_builtin_table',
 '_check_inplace_setting',
 '_check_is_chained_assignment_possible',
 '_check_label_or_level_ambiguity',
 '_check_percentile',
 '_check_setitem_copy',
 '_clear_item_cache',
 '_clip_with_one_bound',
 '_clip_with_scalar',
 '_combine_const',
 '_combine_frame',
 '_combine_match_columns',
 '_combine_match_index',
 '_consolidate',
 '_consolidate_inplace',
 '_construct_axes_dict',
 '_construct_axes_dict_from',
 '_construct_axes_from_arguments',
 '_constructor',
 '_constructor_expanddim',
 '_constructor_sliced',
 '_convert',
 '_count_level',
 '_create_indexer',
 '_cython_table',
 '_data',
 '_deprecations',
 '_dir_additions',
 '_dir_deletions',
 '_drop_axis',
 '_drop_labels_or_levels',
 '_ensure_valid_index',
 '_find_valid_index',
 '_from_arrays',
 '_from_axes',
 '_get_agg_axis',
 '_get_axis',
 '_get_axis_name',
 '_get_axis_number',
 '_get_axis_resolvers',
 '_get_block_manager_axis',
 '_get_bool_data',
 '_get_cacher',
 '_get_index_resolvers',
 '_get_item_cache',
 '_get_label_or_level_values',
 '_get_numeric_data',
 '_get_space_character_free_column_resolvers',
 '_get_value',
 '_get_values',
 '_getitem_bool_array',
 '_getitem_frame',
 '_getitem_multilevel',
 '_gotitem',
 '_iget_item_cache',
 '_indexed_same',
 '_info_axis',
 '_info_axis_name',
 '_info_axis_number',
 '_info_repr',
 '_init_mgr',
 '_internal_get_values',
 '_internal_names',
 '_internal_names_set',
 '_is_builtin_func',
 '_is_cached',
 '_is_copy',
 '_is_cython_func',
 '_is_datelike_mixed_type',
 '_is_homogeneous_type',
 '_is_label_or_level_reference',
 '_is_label_reference',
 '_is_level_reference',
 '_is_mixed_type',
 '_is_numeric_mixed_type',
 '_is_view',
 '_ix',
 '_ixs',
 '_join_compat',
 '_maybe_cache_changed',
 '_maybe_update_cacher',
 '_metadata',
 '_needs_reindex_multi',
 '_obj_with_exclusions',
 '_protect_consolidate',
 '_reduce',
 '_reindex_axes',
 '_reindex_columns',
 '_reindex_index',
 '_reindex_multi',
 '_reindex_with_indexers',
 '_repr_data_resource_',
 '_repr_fits_horizontal_',
 '_repr_fits_vertical_',
 '_repr_html_',
 '_repr_latex_',
 '_reset_cache',
 '_reset_cacher',
 '_sanitize_column',
 '_selected_obj',
 '_selection',
 '_selection_list',
 '_selection_name',
 '_series',
 '_set_as_cached',
 '_set_axis',
 '_set_axis_name',
 '_set_is_copy',
 '_set_item',
 '_set_value',
 '_setitem_array',
 '_setitem_frame',
 '_setitem_slice',
 '_setup_axes',
 '_shallow_copy',
 '_slice',
 '_stat_axis',
 '_stat_axis_name',
 '_stat_axis_number',
 '_to_dict_of_blocks',
 '_try_aggregate_string_function',
 '_typ',
 '_unpickle_frame_compat',
 '_unpickle_matrix_compat',
 '_update_inplace',
 '_validate_dtype',
 '_values',
 '_where',
 '_xs',
 'abs',
 'add',
 'add_prefix',
 'add_suffix',
 'agg',
 'aggregate',
 'align',
 'all',
 'any',
 'append',
 'apply',
 'applymap',
 'as_matrix',
 'asfreq',
 'asof',
 'assign',
 'astype',
 'at',
 'at_time',
 'axes',
 'between_time',
 'bfill',
 'bool',
 'boxplot',
 'clip',
 'clip_lower',
 'clip_upper',
 'columns',
 'combine',
 'combine_first',
 'compound',
 'copy',
 'corr',
 'corrwith',
 'count',
 'cov',
 'cummax',
 'cummin',
 'cumprod',
 'cumsum',
 'degree_p',
 'degree_t',
 'describe',
 'diff',
 'div',
 'divide',
 'dot',
 'drop',
 'drop_duplicates',
 'droplevel',
 'dropna',
 'dtypes',
 'duplicated',
 'empty',
 'eq',
 'equals',
 'etest_p',
 'eval',
 'ewm',
 'expanding',
 'explode',
 'ffill',
 'fillna',
 'filter',
 'first',
 'first_valid_index',
 'floordiv',
 'from_dict',
 'from_records',
 'ge',
 'gender',
 'get',
 'get_dtype_counts',
 'get_ftype_counts',
 'get_values',
 'groupby',
 'gt',
 'head',
 'hist',
 'hsc_b',
 'hsc_p',
 'hsc_s',
 'iat',
 'idxmax',
 'idxmin',
 'iloc',
 'index',
 'infer_objects',
 'info',
 'insert',
 'interpolate',
 'isin',
 'isna',
 'isnull',
 'items',
 'iteritems',
 'iterrows',
 'itertuples',
 'join',
 'keys',
 'kurt',
 'kurtosis',
 'last',
 'last_valid_index',
 'le',
 'loc',
 'lookup',
 'lt',
 'mad',
 'mask',
 'max',
 'mba_p',
 'mean',
 'median',
 'melt',
 'memory_usage',
 'merge',
 'min',
 'mod',
 'mode',
 'mul',
 'multiply',
 'ndim',
 'ne',
 'nlargest',
 'notna',
 'notnull',
 'nsmallest',
 'nunique',
 'pct_change',
 'pipe',
 'pivot',
 'pivot_table',
 'plot',
 'pop',
 'pow',
 'prod',
 'product',
 'quantile',
 'query',
 'radd',
 'rank',
 'rdiv',
 'reindex',
 'reindex_like',
 'rename',
 'rename_axis',
 'reorder_levels',
 'replace',
 'resample',
 'reset_index',
 'rfloordiv',
 'rmod',
 'rmul',
 'rolling',
 'round',
 'rpow',
 'rsub',
 'rtruediv',
 'salary',
 'sample',
 'select_dtypes',
 'sem',
 'set_axis',
 'set_index',
 'shape',
 'shift',
 'size',
 'skew',
 'sl_no',
 'slice_shift',
 'sort_index',
 'sort_values',
 'sparse',
 'specialisation',
 'squeeze',
 'ssc_b',
 'ssc_p',
 'stack',
 'status',
 'std',
 'style',
 'sub',
 'subtract',
 'sum',
 'swapaxes',
 'swaplevel',
 'tail',
 'take',
 'to_clipboard',
 'to_csv',
 'to_dense',
 'to_dict',
 'to_excel',
 'to_feather',
 'to_gbq',
 'to_hdf',
 'to_html',
 'to_json',
 'to_latex',
 'to_msgpack',
 'to_numpy',
 'to_parquet',
 'to_period',
 'to_pickle',
 'to_records',
 'to_sparse',
 'to_sql',
 'to_stata',
 'to_string',
 'to_timestamp',
 'to_xarray',
 'transform',
 'transpose',
 'truediv',
 'truncate',
 'tshift',
 'tz_convert',
 'tz_localize',
 'unstack',
 'update',
 'values',
 'var',
 'where',
 'workex',
 'xs']
```

In [8]:
```
dataset["ssc_p"].dtypes
```

Out[8]:
```
dtype('float64')
```

In [9]:
```
for columnName in dataset.columns:
    print(columnName)
```

```
sl_no
gender
ssc_p
ssc_b
hsc_p
hsc_b
hsc_s
degree_p
degree_t
workex
etest_p
specialisation
mba_p
status
salary
```

In [10]:
```
if(dataset["ssc_p"].dtype=='O'):
    print("qual")
else:
    print("quan")
```

```
quan
```

In [11]:
```
quan=[]
qual=[]
for columnName in dataset.columns:
    #print(columnName)
    if(dataset[columnName].dtype=='O'):
        #print("qual")
        qual.append(columnName)
    else:
        #print("quan")
        quan.append(columnName)
```

In [12]:
```
quan
```

Out[12]:
```
['sl_no', 'ssc_p', 'hsc_p', 'degree_p', 'etest_p', 'mba_p', 'salary']
```

In [13]:
```
qual
```

Out[13]:
```
['gender',
 'ssc_b',
 'hsc_b',
 'hsc_s',
 'degree_t',
 'workex',
 'specialisation',
 'status']
```

In [14]:
```
def quanQual(dataset):
    quan=[]
    qual=[]
    for columnName in dataset.columns:
        #print(columnName)
        if(dataset[columnName].dtype=='O'):
            #print("qual")
            qual.append(columnName)
        else:
            #print("quan")
            quan.append(columnName)
    return quan,qual
```

In [15]:
```
quanQual(dataset)
```

Out[15]:
```
(['sl_no', 'ssc_p', 'hsc_p', 'degree_p', 'etest_p', 'mba_p', 'salary'],
 ['gender',
  'ssc_b',
  'hsc_b',
  'hsc_s',
  'degree_t',
  'workex',
  'specialisation',
  'status'])
```

In [16]:
```
quan,qual=quanQual(dataset)
```

In [17]:
```
quan
```

Out[17]:
```
['sl_no', 'ssc_p', 'hsc_p', 'degree_p', 'etest_p', 'mba_p', 'salary']
```

In [18]:
```
qual
```

Out[18]:
```
['gender',
 'ssc_b',
 'hsc_b',
 'hsc_s',
 'degree_t',
 'workex',
 'specialisation',
 'status']
```

In [19]:
```
dataset
```

Out[19]:

	sl_no	gender	ssc_p	ssc_b	hsc_p	hsc_b	hsc_s	degree_p	degree_t	workex	etest_p	specialisation	mba_p	status	salary
0	1	M	67.00	Others	91.00	Others	Commerce	58.00	Sci&Tech	No	55.0	Mkt&HR	58.80	Placed	270000.0
1	2	M	79.33	Central	78.33	Others	Science	77.48	Sci&Tech	Yes	86.5	Mkt&Fin	66.28	Placed	200000.0
2	3	M	65.00	Central	68.00	Central	Arts	64.00	Comm&Mgmt	No	75.0	Mkt&Fin	57.80	Placed	250000.0
3	4	M	56.00	Central	52.00	Central	Science	52.00	Sci&Tech	No	66.0	Mkt&HR	59.43	Not Placed	NaN
4	5	M	85.80	Central	73.60	Central	Commerce	73.30	Comm&Mgmt	No	96.8	Mkt&Fin	55.50	Placed	425000.0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
210	211	M	80.60	Others	82.00	Others	Commerce	77.60	Comm&Mgmt	No	91.0	Mkt&Fin	74.49	Placed	400000.0
211	212	M	58.00	Others	60.00	Others	Science	72.00	Sci&Tech	No	74.0	Mkt&Fin	53.62	Placed	275000.0
212	213	M	67.00	Others	67.00	Others	Commerce	73.00	Comm&Mgmt	Yes	59.0	Mkt&Fin	69.72	Placed	295000.0
213	214	F	74.00	Others	66.00	Others	Commerce	58.00	Comm&Mgmt	No	70.0	Mkt&HR	60.23	Placed	204000.0
214	215	M	62.00	Central	58.00	Others	Science	53.00	Comm&Mgmt	No	89.0	Mkt&HR	60.22	Not Placed	NaN

215 rows × 15 columns

Central tendency

In [20]:
```
dataset["ssc_p"].mean()
```

Out[20]:
```
67.30339534883721
```

In [21]:
```
dataset["ssc_p"].median()
```

Out[21]:
```
67.0
```

In [22]:
```
dataset["ssc_p"].mode()[0]
```

Out[22]:
```
62.0
```

In [23]:
```
dataset.describe()
```

Out[23]:

	sl_no	ssc_p	hsc_p	degree_p	etest_p	mba_p	salary
count	215.000000	215.000000	215.000000	215.000000	215.000000	215.000000	148.000000
mean	108.000000	67.303395	66.333163	66.370186	72.100558	62.278186	288655.405405
std	62.209324	10.827205	10.897509	7.358743	13.275956	5.833385	93457.452420
min	1.000000	40.890000	37.000000	50.000000	50.000000	51.210000	200000.000000
25%	54.500000	60.600000	60.900000	61.000000	60.000000	57.945000	240000.000000
50%	108.000000	67.000000	65.000000	66.000000	71.000000	62.000000	265000.000000
75%	161.500000	75.700000	73.000000	72.000000	83.500000	66.255000	300000.000000
max	215.000000	89.400000	97.700000	91.000000	98.000000	77.890000	940000.000000

In [24]:
```
descriptive=pd.DataFrame(index=["Mean","Median","Mode"],columns=quan)
```

In [25]:
```
descriptive["sl_no"]["Mean"]
```

Out[25]:
```
nan
```

In [26]:
```
descriptive=pd.DataFrame(index=["Mean","Median","Mode"],columns=quan)
for columnName in quan:
    descriptive[columnName]["Mean"]=dataset[columnName].mean()
```

In [27]:
```
descriptive
```

Out[27]:

	sl_no	ssc_p	hsc_p	degree_p	etest_p	mba_p	salary
Mean	108	67.3034	66.3332	66.3702	72.1006	62.2782	288655
Median	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Mode	NaN	NaN	NaN	NaN	NaN	NaN	NaN

In [28]:
```
descriptive=pd.DataFrame(index=["Mean","Median","Mode"],columns=quan)
for columnName in quan:
    descriptive[columnName]["Mean"]=dataset[columnName].mean()
    descriptive[columnName]["Median"]=dataset[columnName].median()
    descriptive[columnName]["Mode"]=dataset[columnName].mode()[0]
```

In [29]:
```
descriptive
```

Out[29]:

	sl_no	ssc_p	hsc_p	degree_p	etest_p	mba_p	salary
Mean	108	67.3034	66.3332	66.3702	72.1006	62.2782	288655
Median	108	67	65	66	71	62	265000
Mode	1	62	63	65	60	56.7	300000

In [30]:
```
import numpy as np
```

In [33]:
```
np.percentile(dataset["ssc_p"],50)
```

Out[33]:
```
67.0
```

In [32]:
```
dataset.describe()
```

Out[32]:

	sl_no	ssc_p	hsc_p	degree_p	etest_p	mba_p	salary
count	215.000000	215.000000	215.000000	215.000000	215.000000	215.000000	148.000000
mean	108.000000	67.303395	66.333163	66.370186	72.100558	62.278186	288655.405405
std	62.209324	10.827205	10.897509	7.358743	13.275956	5.833385	93457.452420
min	1.000000	40.890000	37.000000	50.000000	50.000000	51.210000	200000.000000
25%	54.500000	60.600000	60.900000	61.000000	60.000000	57.945000	240000.000000
50%	108.000000	67.000000	65.000000	66.000000	71.000000	62.000000	265000.000000
75%	161.500000	75.700000	73.000000	72.000000	83.500000	66.255000	300000.000000
max	215.000000	89.400000	97.700000	91.000000	98.000000	77.890000	940000.000000

In [44]:
```
descriptive=pd.DataFrame(index=["Mean","Median","Mode","25%","50%","75%","100%"],columns=quan)
for columnName in quan:
    descriptive[columnName]["Mean"]=dataset[columnName].mean()
    descriptive[columnName]["Median"]=dataset[columnName].median()
    descriptive[columnName]["Mode"]=dataset[columnName].mode()[0]
    descriptive[columnName]["25%"]=dataset.describe()[columnName]["25%"]
    descriptive[columnName]["50%"]=dataset.describe()[columnName]["50%"]
    descriptive[columnName]["75%"]=dataset.describe()[columnName]["75%"]
    descriptive[columnName]["100%"]=dataset.describe()[columnName]["max"]
```

In [45]:
```
descriptive
```

Out[45]:

	sl_no	ssc_p	hsc_p	degree_p	etest_p	mba_p	salary
Mean	108	67.3034	66.3332	66.3702	72.1006	62.2782	288655
Median	108	67	65	66	71	62	265000
Mode	1	62	63	65	60	56.7	300000
25%	54.5	60.6	60.9	61	60	57.945	240000
50%	108	67	65	66	71	62	265000
75%	161.5	75.7	73	72	83.5	66.255	300000
100%	215	89.4	97.7	91	98	77.89	940000

In [47]:
```
descriptive=pd.DataFrame(index=["Mean","Median","Mode","Min","25%","50%","75%","100%"],columns=quan)
for columnName in quan:
    descriptive[columnName]["Mean"]=dataset[columnName].mean()
    descriptive[columnName]["Median"]=dataset[columnName].median()
    descriptive[columnName]["Mode"]=dataset[columnName].mode()[0]
    descriptive[columnName]["Min"]=dataset[columnName].min()
    descriptive[columnName]["25%"]=dataset.describe()[columnName]["25%"]
    descriptive[columnName]["50%"]=dataset.describe()[columnName]["50%"]
    descriptive[columnName]["75%"]=dataset.describe()[columnName]["75%"]
    descriptive[columnName]["100%"]=dataset.describe()[columnName]["max"]
```

In [48]:
```
descriptive
```

Out[48]:

	sl_no	ssc_p	hsc_p	degree_p	etest_p	mba_p	salary
Mean	108	67.3034	66.3332	66.3702	72.1006	62.2782	288655
Median	108	67	65	66	71	62	265000
Mode	1	62	63	65	60	56.7	300000
Min	1	40.89	37	50	50	51.21	200000
25%	54.5	60.6	60.9	61	60	57.945	240000
50%	108	67	65	66	71	62	265000
75%	161.5	75.7	73	72	83.5	66.255	300000
100%	215	89.4	97.7	91	98	77.89	940000

In [58]:
```
descriptive=pd.DataFrame(index=["Mean","Median","Mode","Min","25%","50%","75%","100%","IQR",
                               "1.5IQR","LesserRange","GreaterRange"],columns=quan)
for columnName in quan:
    descriptive[columnName]["Mean"]=dataset[columnName].mean()
    descriptive[columnName]["Median"]=dataset[columnName].median()
    descriptive[columnName]["Mode"]=dataset[columnName].mode()[0]
    descriptive[columnName]["Min"]=dataset[columnName].min()
    descriptive[columnName]["25%"]=dataset.describe()[columnName]["25%"]
    descriptive[columnName]["50%"]=dataset.describe()[columnName]["50%"]
    descriptive[columnName]["75%"]=dataset.describe()[columnName]["75%"]
    descriptive[columnName]["100%"]=dataset.describe()[columnName]["max"]
    descriptive[columnName]["IQR"]=descriptive[columnName]["75%"]-descriptive[columnName]["25%"]
    descriptive[columnName]["1.5IQR"]=1.5*descriptive[columnName]["IQR"]
    descriptive[columnName]["LesserRange"]=descriptive[columnName]["25%"]- descriptive[columnName]["1.5IQR"]
    descriptive[columnName]["GreaterRange"]=descriptive[columnName]["75%"]+ descriptive[columnName]["1.5IQR"]
```

In [59]:
```
descriptive
```

Out[59]:

	sl_no	ssc_p	hsc_p	degree_p	etest_p	mba_p	salary
Mean	108	67.3034	66.3332	66.3702	72.1006	62.2782	288655
Median	108	67	65	66	71	62	265000
Mode	1	62	63	65	60	56.7	300000
Min	1	40.89	37	50	50	51.21	200000
25%	54.5	60.6	60.9	61	60	57.945	240000
50%	108	67	65	66	71	62	265000
75%	161.5	75.7	73	72	83.5	66.255	300000
100%	215	89.4	97.7	91	98	77.89	940000
IQR	107	15.1	12.1	11	23.5	8.31	60000
1.5IQR	160.5	22.65	18.15	16.5	35.25	12.465	90000
LesserRange	-106	37.95	42.75	44.5	24.75	45.48	150000
GreaterRange	322	98.35	91.15	88.5	118.75	78.72	390000

In [60]:
```
if(descriptive["ssc_p"]["LesserRange"]>descriptive["ssc_p"]["Min"]):
    print("LesserRange Outlier")
else:
    print("No LR Outlier")
```

```
No LR Outlier
```

In [63]:
```
lesser=[]
greater=[]

for columnName in quan:
    if(descriptive[columnName]["LesserRange"]>descriptive[columnName]["Min"]):
        lesser.append(columnName)
    if(descriptive[columnName]["GreaterRange"]<descriptive[columnName]["100%"]):
        greater.append(columnName)
```

In [64]:
```
lesser
```

Out[64]:
```
['hsc_p']
```

In [65]:
```
greater
```

Out[65]:
```
['hsc_p', 'degree_p', 'salary']
```

In [68]:
```
for lesscolumn in lesser:
    dataset[lesscolumn][dataset[lesscolumn]<descriptive[lesscolumn]["LesserRange"]]=descriptive[lesscolumn]["LesserRange"]
for greatcolumn in greater:
    dataset[greatcolumn][dataset[greatcolumn]>descriptive[greatcolumn]["GreaterRange"]]=descriptive[greatcolumn]["GreaterRange"]
```

```
C:\Anaconda3\envs\aiml\lib\site-packages\ipykernel_launcher.py:2: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame

See the caveats in the documentation: http://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

C:\Anaconda3\envs\aiml\lib\site-packages\ipykernel_launcher.py:4: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame

See the caveats in the documentation: http://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
  after removing the cwd from sys.path.
```

In [70]:
```
descriptive=pd.DataFrame(index=["Mean","Median","Mode","Min","25%","50%","75%","100%","IQR",
                               "1.5IQR","LesserRange","GreaterRange"],columns=quan)
for columnName in quan:
    descriptive[columnName]["Mean"]=dataset[columnName].mean()
    descriptive[columnName]["Median"]=dataset[columnName].median()
    descriptive[columnName]["Mode"]=dataset[columnName].mode()[0]
    descriptive[columnName]["Min"]=dataset[columnName].min()
    descriptive[columnName]["25%"]=dataset.describe()[columnName]["25%"]
    descriptive[columnName]["50%"]=dataset.describe()[columnName]["50%"]
    descriptive[columnName]["75%"]=dataset.describe()[columnName]["75%"]
    descriptive[columnName]["100%"]=dataset.describe()[columnName]["max"]
    descriptive[columnName]["IQR"]=descriptive[columnName]["75%"]-descriptive[columnName]["25%"]
    descriptive[columnName]["1.5IQR"]=1.5*descriptive[columnName]["IQR"]
    descriptive[columnName]["LesserRange"]=descriptive[columnName]["25%"]- descriptive[columnName]["1.5IQR"]
    descriptive[columnName]["GreaterRange"]=descriptive[columnName]["75%"]+ descriptive[columnName]["1.5IQR"]
```

In [71]:
```
lesser=[]
greater=[]

for columnName in quan:
    if(descriptive[columnName]["LesserRange"]>descriptive[columnName]["Min"]):
        lesser.append(columnName)
    if(descriptive[columnName]["GreaterRange"]<descriptive[columnName]["100%"]):
        greater.append(columnName)
```

In [72]:
```
lesser
```

Out[72]:
```
[]
```

In [73]:
```
greater
```

Out[73]:
```
[]
```

In [74]:
```
descriptive
```

Out[74]:

	sl_no	ssc_p	hsc_p	degree_p	etest_p	mba_p	salary
Mean	108	67.3034	66.3347	66.3586	72.1006	62.2782	277649
Median	108	67	65	66	71	62	265000
Mode	1	62	63	65	60	56.7	300000
Min	1	40.89	42.75	50	50	51.21	200000
25%	54.5	60.6	60.9	61	60	57.945	240000
50%	108	67	65	66	71	62	265000
75%	161.5	75.7	73	72	83.5	66.255	300000
100%	215	89.4	91.15	88.5	98	77.89	390000
IQR	107	15.1	12.1	11	23.5	8.31	60000
1.5IQR	160.5	22.65	18.15	16.5	35.25	12.465	90000
LesserRange	-106	37.95	42.75	44.5	24.75	45.48	150000
GreaterRange	322	98.35	91.15	88.5	118.75	78.72	390000

```
In [ ]:
```